Chapter 1

Overview

X16 Open is an alternative open source operating system for the Commander X16.

It is built around these ideas:

- **65C02 only:** The system can be simplified by supporting only one CPU, the 65C02.
- **Single user, single process:** The 65C02 CPU was designed with single user, single process systems in mind.
- **Everything is a file:** Devices are accessed through file handles, using common functions such as open, read, write and close.
- **The kernel is thin:** The kernel offloads as much as possible of the work to device drivers. Drivers are modular and can be loaded at runtime, while some drivers are embedded into the kernel.

Chapter 2

The Kernel

- Kernel API
- Device Table
- File Table

2.1 Introduction

Most of the work in the system is done by device drivers. The kernel is just a thin layer that provides a common interface to the drivers.

The core device drivers required to start the system are embedded into the kernel ROM bank (bank 0). Other device drivers, including alternatives to the core drivers, can be stored in ROM banks 1-255 or even be loaded into RAM at runtime.

You are free to decide what device drivers are used for different tasks. If you, for example, want a USB keyboard instead of the default PS/2 keyboard, it is possible to load and use a custom USB keyboard device driver instead of the default PS/2 keyboard driver.

The kernel is inspired by Unix, where almost everything is a file. The device driver interface is, consequently, based on functions such as open, read, write, and close.

2.2 API Functions

The kernel's internal code and data structures are subject to change. Only the functions listed in this section are part of the stable interface.

Name	Address	Description
reset	$c000	System reset
alloc	$c003	Allocates one RAM bank (8 kB)
free	$c006	Frees one RAM bank
device_register	$c009	Registers a new device driver
device_unregister	$c00c	Unregisters a device
file_open	$c00f	Opens a file
file_dup	$c012	Duplicates a file table entry
file_read_byte	$c015	Reads one byte from a file

Continued on next page

Name	Address	Description
file_read_block	$c018	Reads a block of bytes from a file
file_write_byte	$c01b	Writes one byte to a file
file_write_block	$c01e	Writes a block of bytes to a file
file_seek	$c021	Seeks a position in a file
file_position	$c024	Returns current position in a file
file_close	$c027	Closes a file
file_ioctl	$c02a	Driver-specific control commands
stdin_read	$c02d	Reads one byte from stdin
stdout_write	$c030	Writes one byte to stdout
get_ticks	$c033	Returns the jiffy clock
set_ticks	$c036	Sets the jiffy clock
get_errmsg	$c039	Returns system error message
set_errmsg	$c03c	Sets system error message
sys_info	$c03f	Get system info

2.3 Passing and Returning Values

Values are passed to the kernel functions and returned through the A, X, Y registers, and 14 zero page registers named r0-13 (address $02-$0f).

Multi-byte values are passed or returned least significant byte first (little-endian).

2.4 Error Handling

All kernel functions must indicate an error condition by setting the carry flag on return.

An error code is always returned in the A register. The A register can return other values if no error occurred. A plain text error message is available through the get_errmsg function.

List of error codes:

Code	Name	Description
0	EOK	No error
1	ENOMEM	Out of memory
2	EMEMINUSE	Requested memory already in use
3	ENODEVSPC	Too many devices
4	ENODEV	Device does not exist
5	EDEVBUSY	Device is in use
6	ENOFILESPC	Too many files
7	ENOTFOUND	File not found
8	ENOTFILE	Path is not a file
9	ENOTDIR	Path is not a directory
10	ENOTOPEN	File not open
11	EFILEOPEN	File is open
12	EINVAL	Invalid argument or operation
13	EIO	Generic driver failure
14	EEOF	End of file
255	ECUSTOM	Custom error message

2.5 Device Table

The kernel uses a device table to keep track of device drivers. The device table has room for 16 devices, and contains the following fields:

Field	Values	Description
addr	0-65535	Device driver address
bank	0-255	Device driver bank
flags	0-255	See list of flags below

The flags stored in the device table are:

- Bit 0: Set if reading is supported.
- Bit 1: Set if writing is supported.
- Bit 2: Set if appending is supported.
- Bit 3: Set if seeking is supported.
- Bit 4: Set if a block device, unset if a char device.
- Bits 5-6: Driver location, where 00 = Not present, 01 = ROM bank 0 or low RAM, 10 = ROM bank 1-255, 11 = RAM bank 0-255.
- Bit 7: Set if kernel is requested to call the driver's on_irq function on every IRQ.

A device driver is added to the device table when you call the device_register function. The device driver is removed when you call device_unregister. It is not recommended or safe to manipulate the device table directly.

2.6 File Table

The kernel has a file table to keep track of all opened files. There is room for 16 files in the file table. The table holds the following information:

Field	Values	Description
driver_id	0-15	Driver's index in device table

Continued on next page

Field	Values	Description
context_id	0-255	The context ID is used internally by drivers to keep track of multiple open files
flags	0-255	See list of flags below

The flags in the file table are:

- Bit 0: Set if opened for reading.
- Bit 1: Set if opened for writing.
- Bit 2: Set if opened for appending.
- Bit 3: Set if a file, unset if a directory.
- Bits 4-6: Reserved.
- Bit 7: Set if file is open.

A file is added to the file table when you call the file_open function, and it is removed when you call the file_close function. It is not recommended or safe to manipulate the file table directly.

Some file handles have a fixed meaning to the system:

- 0: STDIN.
- 1: STDOUT.
- 2: Keyboard.
- 3: Mouse.
- 4: SNES game controllers.

2.7 API Function Specifications

2.7.1 Function: reset

Call address: $c000
Input: None

Output: None
Errors: None

This function does a cold reset of the system. It is called automatically when the system is reset. It is also possible to call the function while running the system.

The function frees all banked RAM and initializes the core device drivers:

0. X16 terminal driver.
1. Generic I2C driver.
2. PS/2 keyboard driver.
3. PS/2 mouse driver.
4. SNES game controller driver.
5. SD card driver.

2.7.2 Function: alloc

Call address: $c003
Input: A = RAM bank
Output: A = RAM bank
Errors: ENOMEM, EMEMINUSE

The X16 has no hardware memory management. The kernel's memory management functions (alloc and free) keep track of which part of banked RAM is free to use. All well-behaved applications should use these functions. They may otherwise overwrite custom device drivers loaded into banked RAM.

RAM bank 0 is used by the kernel and can't be allocated for other purposes.

If this function is called with A = 0, it will search for and allocate the next available RAM bank. Error ENOMEM is set if there is no free memory.

If the function is called with A > 0, it will attempt to allocate the specified RAM bank. EMEMINUSE is returned if that bank is already allocated.

A successfully allocated RAM bank is returned in the A register.

2.7.3 Function: free

Call address: $c006
Input: A = RAM bank
Output: None
Errors: EINVAL

This function frees a RAM bank that was allocated with the alloc function.

RAM bank 0 is used by the system and may not be freed. If this function is called with A = 0, the EINVAL error is returned.

2.7.4 Function: device_register

Call address: $c009
Input: X = Device ID, r0-1 = Driver address, r2 = Driver bank
Output: X = Device ID
Errors: ENODEVSPC, EDEVBUSY, EIO

The function registers a device driver that is already present in memory.

The system's base drivers needed to boot the system are embedded into the kernel ROM bank, and are automatically registered and initialized by the reset function.

If called with X = 255, the first free device ID will be used. Otherwise, the function attempts to use the specified device ID.

The driver's on_register function is automatically invoked when calling this function. The kernel's device table is updated with information on the new driver, including flags returned by the on_register function.

The ENODEVSPC error is returned if the driver table is full.

The EDEVBUSY error is returned if the specified device ID is already in use. In that case you must first call device_unregister to free the current driver.

EIO is a driver-specific error.

2.7.5 Function: device_unregister

Call address: $c00c
Input: X = Device ID
Output: None
Errors: EDEVBUSY, EIO

Unregisters the specified device driver.

The EDEVBUSY error is returned if the device driver is linked to an open file.

EIO is a driver-specific error.

2.7.6 Function: file_open

Call address: $c00f
Input: X = File handle, Y = Device ID, A = Flags, r0-1 = Path address, r2 = Path bank
Output: X = File handle, Y = Context ID, A = Flags
Errors: ENODEV, ENOFILESPC, EFILEOPEN, ENOTFILE, ENOTFOUND, EINVAL, EIO

Opens a file using the specified device driver.

If the X register is set to $ff, the new file is linked to the next available file handle. Otherwise, the function attempts to use the specified file handle.

The file is opened using the device driver specified in the Y register.

You must set exactly one of these flags when calling the function. The kernel doesn't support opening a file in multiple modes (or no mode).

- Bit 0: Set if opened for reading.
- Bit 1: Set if opened for writing.
- Bit 2: Set if opened for appending.

The path must be a null-terminated string.

If successfully opened, the file is entered into the file table. The file handle is returned in the X register and the driver's file context ID is

returned in the Y register. The context is managed by the driver, and is used by the driver to keep track of multiple files that are opened at the same time. The file flags are returned in the A register. In addition to the flag set as input to this function, bit 3 is set if a file and unset if a directory, and bit 7 is set if the file was successfully opened.

ENODEV is returned if no driver is linked to the specified device ID.

The ENOFILESPC error is set if the file table is full. There can be at most 16 open files at any given time. The same error is also returned if the number of files the driver can handle is exceeded.

The EFILEOPEN error is returned if you try to link a new file to a file handle that is already in use.

ENOTFILE is returned if attempting to write or append to a directory.

The function fails with ENOTFOUND if the file is not found.

The EINVAL error is returned if the device driver doesn't have the capability requested in the flags argument. The same error is returned if not exactly one flag is set.

EIO is a driver-specific error.

2.7.7 Function: file_dup

Call address: $c012
Input: X = Source file handle, Y = Destination file handle
Output: Y = Destination file handle
Errors: ENOFILESPC, ENOTOPEN, EFILEOPEN, EINVAL, EIO

This function duplicates the file handle X as file handle Y without opening a new file.

The function calls the driver's dup function. If necessary, the driver keeps track of the number of references to a specific file context. A file that has been duplicated will typically not be closed by the driver until all references have been closed.

2.7.8 Function: file_read_byte

Call address: $c015
Input: X = File handle
Output: A = Value
Errors: ENOTOPEN, EINVAL, EEOF, EIO

Reads one byte from the specified file.

The ENOTOPEN error is returned if the specified file handle is not open.

The EINVAL error is set if the file is not opened for reading.

EEOF is returned if the read operation was attempted at end of the file and no byte value was returned. A successful read operation of the final byte does not return EEOF; the caller receives EEOF on the following read attempt.

EIO is a driver-specific error.

2.7.9 Function: file_read_block

Call address: $c018
Input: X = File handle, r0-1 = Requested number of bytes, r2-3 = Buffer address, r4 = Buffer bank
Output: r5-6 = Number of bytes read
Errors: ENOTOPEN, EINVAL, EEOF, EIO

Reads a block from the specified file, at most the requested number of bytes. It is not guaranteed that you get the requested number of bytes.

The ENOTOPEN error is returned if the specified file handle is not open.

The EINVAL error is returned if the file is not opened for reading.

EEOF is set if the read operation was attempted at end of the file and no byte value was returned. A successful read operation of the final byte does not return EEOF; the caller receives EEOF on the next read attempt.

EIO is a driver-specific error.

2.7.10 Function: file_write_byte

Call address: $c01b
Input: X = File handle, A = Byte value
Output: None
Errors: ENOTOPEN, ENOTFILE, EINVAL, EIO

Writes one byte to the specified file.

The ENOTOPEN error is returned if the specified file handle is not open.

ENOTFILE is returned if attempting to write or append to a directory.

The EINVAL error is returned if the file is not opened for writing or appending.

EIO is a driver-specific error.

2.7.11 Function: file_write_block

Call address: $c01e
Input: X = File handle, r0-1 = Buffer size, r2-3 = Buffer address, r4 = Buffer bank
Output: r5-6 = Number of bytes written
Errors: ENOTOPEN, ENOTFILE, EINVAL, EIO

Writes one block to the specified file, at most the entire buffer size passed in r0-1.

The ENOTOPEN error is returned if the specified file handle is not open.

ENOTFILE is returned if attempting to write or append to a directory.

The EINVAL error is returned if the file is not opened for writing or appending.

EIO is a driver-specific error.

2.7.12 Function: file_seek

Call address: $c021
Input: X = File handle, r0-3 = File position

Output: r0-3 = File position
Errors: ENOTOPEN, EINVAL, EIO

Moves the file pointer to the requested position.

The file position after this call is returned in r0-3.

The kernel only allows writing to files in a sequential order at the end of the file, and seeking is thus allowed only on files opened for reading. Attempting to seek on a file opened for writing or appending returns the EINVAL error.

EINVAL is also returned if the device driver doesn't support seeking.

The ENOTOPEN error is returned if the specified file handle is not open.

EIO is a driver-specific error.

2.7.13 Function: file_position

Call address: $c024
Input: X = File handle
Output: r0-3 = File position
Errors: ENOTOPEN, EINVAL, EIO

Returns the current position in the specified file in r0-3.

ENOTOPEN is returned if the file is not open.

EINVAL can be returned if the driver doesn't support reading the file positions.

EIO is a driver-specific error.

2.7.14 Function: file_close

Call address: $c027
Input: X = File handle
Output: None
Errors: EINVAL, ENOTOPEN, EIO

Closes the specified file and removes it from the file table.

ENOTOPEN is returned if the file handle is not open.

EIO is a driver-specific error.

2.7.15 Function: file_ioctl

Call address: $c02a
Input: X = File handle, A = Device type, r0 = Command, r1 = Input length, r2-3 = Input data or input buffer address, r4 = Input buffer bank, r5 = Output max length, r6-7 = Output buffer address, r8 = Output buffer bank
Output: A = Output length
Errors: ENOTOPEN, EINVAL, EIO

This function executes a device-specific control command.

The file handle is given in order to let the command affect a specific file context. Device drivers may, however, ignore the file context if it's not relevant to the command. ENOTOPEN is returned if the file is not open.

The type of device that the command is meant for is specified in the A register. EINVAL is returned if the driver doesn't support commands for the specified device type.

The control command is specified in r0. The two most significant bits are used to flag if the input (r2-4) and output (r6-8) are direct values (max two bytes) or buffer pointers:

- Bit 7: Set if output is a direct value, clear if output is a buffer pointer.
- Bit 6: Set if input is a direct value, clear if input is a buffer pointer.

The input is specified in r0-4, and the output is specified in r5-8.

The function returns the output length in the A register. If the output is a direct value, it is returned in r6-7. Otherwise, the output buffer pointer and bank are returned in r6-8.

EIO is a driver-specific error.

2.7.16 Function: stdin_read

Call address: $c02d
Input: None
Output: A = Byte value
Errors: ENOTOPEN, EINVAL, EEOF, EIO

Reads one byte from STDIN, i.e. file handle 0.

The ENOTOPEN error is returned if STDIN is not open.

The EINVAL error is set if STDIN is not opened for reading.

EEOF is returned if the read operation was attempted at end of the file and no byte value was returned. A successful read operation of the final byte does not return EEOF; the caller receives EEOF on the following read attempt. This doesn't normally happen when STDIN is connected to a keyboard that has an endless stream conceptually.

EIO is a driver-specific error.

2.7.17 Function: stdout_write

Call address: $c030
Input: A = Byte value
Output: None
Errors: ENOTOPEN, EINVAL, EIO

Writes one byte to STDOUT, i.e. file handle 1.

The ENOTOPEN error is returned if STDOUT is not open.

The EINVAL error is returned if STDOUT is not opened for writing or appending.

EIO is a driver-specific error.

2.7.18 Function: get_ticks

Call address: $c033
Input: None
Output: r0-r3 = Counter value
Errors: None

Returns the jiffy clock tick counter that is incremented on every IRQ (60 times per second).

2.7.19 Function: set_ticks

Call address: $c036
Input: r0-r3 = Counter value
Output: None
Errors: None

Sets the jiffy clock tick counter that is incremented on every IRQ (60 times per second).

2.7.20 Function: get_errmsg

Call address: $c039
Input: None
Output: A = Error number, r0-1 = Error message address, r2 = Error message bank
Errors: None

Returns the last error number in the A register and a pointer to the last error message, that is always a null-terminated string.

2.7.21 Function: set_errmsg

Call address: $c03c
Input: A = Error number, r0-1 = Error message address, r2 = Error message bank
Output: None
Errors: None

Sets the error number and message that can be retrieved with the get_errmsg function.

2.7.22 Function: sys_info

Call address: $c03f
Input: None
Output: r0-3 = Kernel version (r0 major, r1 minor, r2 patch), r4-5 =

String address, r6 = String bank
Errors: None

Returns the kernel version, and a plain null-terminated text description of the system.

Chapter 3

Device Drivers

- Introduction
- Driver API

3.1 Introduction

X16 Open is based on loadable device drivers that do most of the work in the system.

Device drivers needed to get the system up and running are embedded into the kernel ROM bank, and are automatically registered and initialized when the system is reset.

All drivers can be replaced at runtime, making it possible to customize the system.

3.2 Device Driver Dispatch Function

All device drivers are required to implement a dispatch function that is placed at the start of the driver's place in memory.

The dispatch function takes a "sub-function" number in the X register. Most sub-functions need more arguments that are passed in the A, Y, and r0-13 registers as described below.

The following sub-functions must be supported:

#	Name	Description
0	on_register	Called when the device driver is registered
2	on_unregister	Called when the device driver is unregistered
4	open	Opens a file
6	dup	Called when a file handle is duplicated
8	read_byte	Reads one byte from a file
10	read_block	Reads one block of bytes from a file
12	write_byte	Writes one byte to a file

Continued on next page

#	Name	Description
14	write_block	Writes a block of bytes to a file
16	seek	Moves the file pointer
18	position	Returns the file pointer
20	close	Closes a file
22	ioctl	Driver control command
24	on_irq	Called on every IRQ
26	driver_string	Returns driver name and version

3.3 Function: on_register

Driver function #: 0
Input: None
Output: A = Flags
Errors: EIO

This function is called by the kernel when registering a device driver. It can, for instance, be used to initialize the driver.

The device driver flags (bits 0-5) are returned in the A register. The possible flags are:

- Bit 0: Set if reading is supported.
- Bit 1: Set if writing is supported.
- Bit 2: Set if appending is supported.
- Bit 3: Set if seeking is supported.
- Bit 4: Set if a block device, unset if a char device.
- Bits 5-6: Reserved, set by the kernel, not the driver.
- Bit 7: Set if kernel must call the driver's on_irq function on every IRQ.

EIO is a driver-specific error.

3.4 Driver Function: on_unregister

Driver function #: 2
Input: None
Output: None
Errors: EIO

This function is called by the kernel when unregistering a device driver. It can, for instance, be used to clean up and free driver resources.

EIO is a driver-specific error.

3.5 Function: open

Function #: 4
Input: A = Flags, r0-1 = Path address, r2 = Path bank
Output: Y = Context ID, A = Flags
Errors: ENOTFOUND, ENOTFILE, EINVAL, EIO

Opens a file or directory at the specified path.

You must select exactly one of the following flags with the A register:

- Bit 0: Set if open for reading
- Bit 1: Set if open for writing
- Bit 2: Set if open for appending

The driver's internal context ID is returned in the Y register if the call is successful.

The A register holds the following flags:

- Bits 0-2: Input preserved.
- Bit 3: Set if a file, unset if a directory.
- Bit 7: Set if opened successfully.

ENOTFOUND is returned if the file is not found.

ENOTFILE is returned if attempting to write or append to a directory.

EINVAL is returned if the operation requested by the input flags is not supported by the device, or if not exactly one of the flags was set.

EIO is a driver-specific error.

3.6 Function: dup

Function #: 6
Input: A = Context ID
Output: None
Errors: ENOTOPEN, EIO

Called by the kernel when a file handle is duplicated without opening a new file. If necessary, the driver is responsible for keeping track of the number of references to each file context.

When calling the close function, the driver will typically decrement the reference counter, and, if 0, close the file.

ENOTOPEN is returned if the context is not active.

EIO is a driver-specific error.

3.7 Function: read_byte

Function #: 8
Input: A = Context ID
Output: A = Byte value
Errors: ENOTOPEN, EEOF, EIO

Reads one byte from a file.

The context ID is used internally by the device driver to keep track of multiple open files.

ENOTOPEN is returned if the context ID is not active.

EEOF is returned if no value can be returned, as the file pointer is at the end of the file. EEOF is not returned when successfully reading the final byte; it is returned on the following read attempt.

EIO is a driver-specific error.

3.8 Function: read_block

Function #: 10
Input: A = Context ID, r0-1 = Requested number of bytes, r2-3 = Buffer address, r4 = Buffer bank
Output: r5-6 = Number of bytes read
Errors: ENOTOPEN, EEOF, EIO

Reads one block of bytes from a file. The requested size of the block is set in r0-1. The block is stored at the memory address and bank set in r2-4.

The size of a block is driver-specific. It is not guaranteed that you receive the requested number of bytes. The number of bytes actually read is returned in r5-6.

The context ID is used internally by the device driver to keep track of multiple open files.

ENOTOPEN is returned if the context ID is not active.

EEOF is returned if no value can be returned, as the file pointer is at the end of the file. EEOF is not returned when successfully reading the final byte; it is returned on the following read attempt.

EIO is a driver-specific error.

3.9 Function: write_byte

Function #: 12
Input: A = Context ID, r0 = Byte value
Output: None
Errors: ENOTOPEN, ENOTFILE, EIO

Writes one byte to a file.

The context ID is used internally by the device driver to keep track of multiple open files.

ENOTOPEN is returned if the context ID is not active.

ENOTFILE is returned if attempting to write or append to a directory.

EIO is a driver-specific error.

3.10 Function: write_block

Function #: 14
Input: A = Context ID, r0-1 = Buffer size, r2-3 = Buffer address, r4 = Buffer bank
Output: r5-6 = Number of bytes written
Errors: ENOTOPEN, ENOTFILE, EIO

Writes one block of bytes to a file.

The size of a block is driver-specific. It is not guaranteed that all requested bytes are written to the file. The number of bytes actually written is returned in r5-6.

The context ID is used internally by the device driver to keep track of multiple open files.

ENOTOPEN is returned if the context ID is not active.

ENOTFILE is returned if attempting to write or append to a directory.

EIO is a driver-specific error.

3.11 Function: seek

Function #: 16
Input: A = Context ID, r0-3 = New position
Output: r0-3 = Current position
Errors: EINVAL, ENOTOPEN, EIO

Moves the file pointer to the requested position.

The file position after this call is returned in r0-3.

Seeking is only supported on files opened for reading. Attempting to seek on a file opened for writing or appending returns the EINVAL error.

EINVAL is also returned if the device driver doesn't support seeking.

The ENOTOPEN error is returned if the specified file context ID is not active.

EIO is a driver-specific error.

3.12 Function: position

Function #: 18
Input: A = Context ID
Output: r0-3 = File position
Errors: EINVAL, ENOTOPEN, EIO

Returns the current position in the specified file in r0-3.

ENOTOPEN is returned if the context ID is not active.

EINVAL can be returned if the driver doesn't support reading the file positions.

EIO is a driver-specific error.

3.13 Function: close

Function #: 20
Input: A = Context ID
Output: None
Errors: ENOTOPEN, EIO

If applicable, the driver should keep record of the number of references made to a specific file context. The reference count is decremented on each call to close. Typically, the file context is closed not until the reference count is 0.

ENOTOPEN is returned if the context ID is not active.

EIO is a driver-specific error.

3.14 Function: ioctl

Function #: 22
Input: A = Context ID, r0 = Device type, r1 = Command, r2 = Input

length, r3-4 = Input data or input buffer address, r5 = Input buffer bank, r6 = Output max length, r7-8 = Output buffer address, r9 = Output buffer bank
Output: A = Output length
Errors: EIO

This function executes a driver-specific control command.

The file context ID is passed in the A register. The driver may ignore the context ID if it is not relevant to the control command.

The device type is passed in r0. A driver for a specific device type is expected to handle certain control commands.

The control command is specified in r1. The two most significant bits are used to flag if the input (r3-5) and output (r7-9) are direct values or buffer pointers. Direct values can be at most two bytes long.

- Bit 7: Set if output is a direct value, clear if output is a buffer pointer.
- Bit 6: Set if input is a direct value, clear if input is a buffer pointer.

The input is specified in r2-5:

- r2: Input length.
- r3-4: A direct value (r1 bit 6 set) set by the driver or input buffer address set by the caller (r1 bit 6 clear).
- r5: Input buffer bank set by the caller (r1 bit 6 clear)

The output is specified in r6-9 and the A register:

- r6: Output max length set by the caller.
- r7-8: A direct value (r1 bit 7 set) set by the driver or output buffer address set by the caller (r1 bit 7 clear).
- r9: Output buffer bank set by the caller (r1 bit 7 clear).
- A: Output length set by the driver.

EIO is a driver-specific error.

3.15 Function: on_irq

Function #: 24
Input: None
Output: None
Errors: EIO

This function is only intended to be called by the kernel's IRQ handler.

It can be used by device drivers that need to be updated once every interrupt, for instance terminals.

EIO is a driver-specific error.

3.16 Function: driver_string

Function #: 26
Input: None
Output: r0-1 = String address, r2 = String bank
Errors: None

This function returns a pointer to a null-terminated string that holds a descriptive name for the driver, for instance "x16-terminal v 1.0".